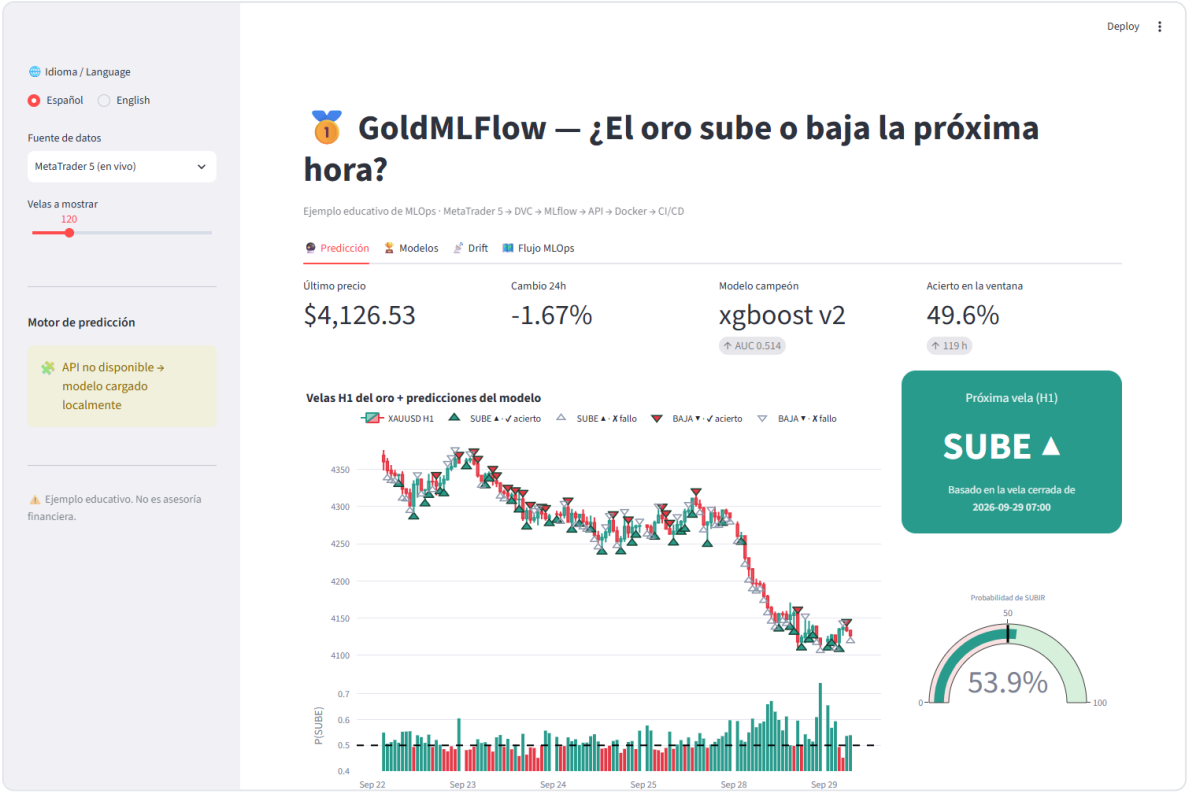
GOLDMLFLOW

MLOps for Dummies

Predicting whether gold goes UP or DOWN in the next hour — the whole MLOps flow, step by step

MetaTrader 5 → DVC → MLflow → FastAPI → Docker → GitHub Actions → Streamlit



Code: github.com/MirandaCR/GoldMLFlow · 2026-09-28
Educational project · Not financial advice

📚 Contents

00 · The Big Picture — What is MLOps?

01 · Setup & run everything

02 · Extract gold data from MetaTrader 5

03 · Data Version Control (DVC)

04 · Validate data & build features

05 · Train models & track them with MLflow

06 · Evaluate, quality gate & Model Registry

07 · Model API (FastAPI)

08 · Containers (Docker)

09 · CI/CD with GitHub Actions

10 · Monitoring & data drift

11 · Streamlit dashboard — see and use the model

12 · From notebook to production code

13 · Glossary & FAQ

00 · The Big Picture — What is MLOps?

🤔 The problem MLOps solves

Training a model in a notebook is like **cooking one great dish at home**. MLOps is **running a restaurant** that serves that dish every day, with the same taste, safely and on time, and changes the recipe when customers' tastes change.

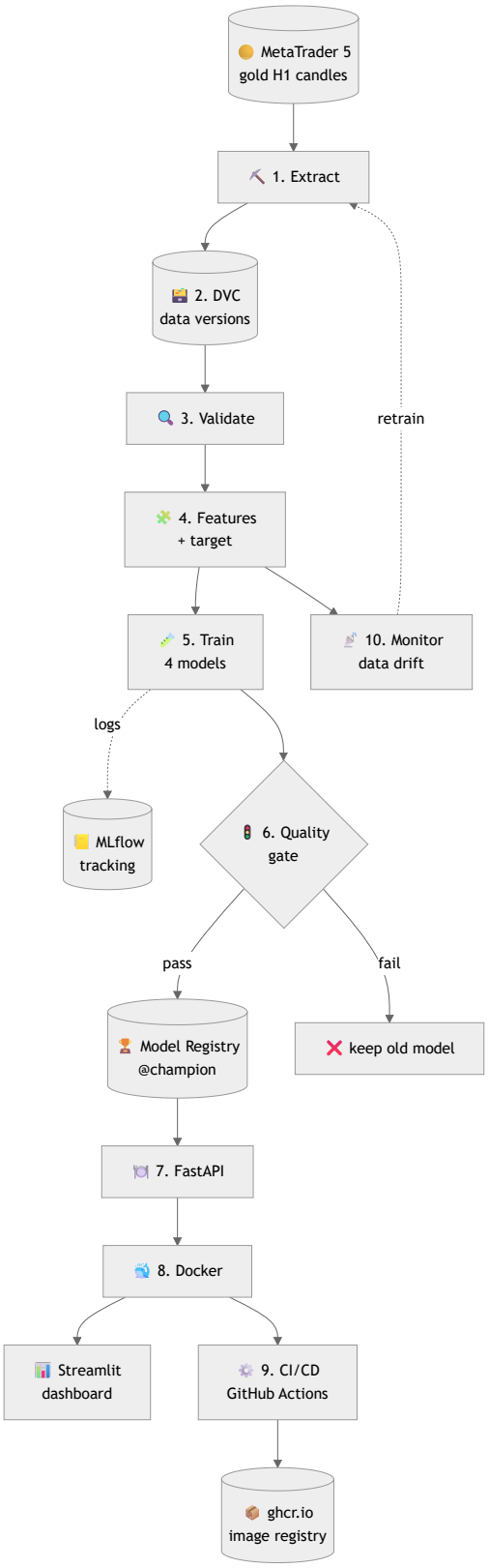
At home (just a notebook) 🏠	Restaurant (MLOps) 🍽️
"Where did I get that data?"	Data is **versioned** (DVC): you can go back to any version
"Which settings gave 0.51?"	Every experiment is **logged** (MLflow)
"It works on my laptop"	It's **packed in a container** (Docker) and runs anywhere
"I copied the model by email"	The model is served through an **API**
"I hope I didn't break anything"	**Automatic tests** on every change (CI)
"I deploy by hand on Fridays"	**Automatic delivery** when tests pass (CD)
"The model got worse... when?"	**Monitoring** warns when the market changes (drift)

🥇 Our example in one sentence

> **Every hour, predict whether the next H1 candle of gold (XAUUSD) will close UP (1) or DOWN (0)**, using data from MetaTrader 5, and run everything around it the MLOps way.

It's a **binary classification** problem on a **time series**.

🗺️ The full flow



🍳 The restaurant analogy (keep it in mind for every chapter)

Stage	Tool	Restaurant analogy
Extract	MetaTrader 5 + Python	🛒 Buying the ingredients at the market
Version data	**DVC**	📸 Dated photos of your pantry: you can always go back
Validate	pandas checks	🔍 Checking the ingredients aren't rotten
Features	pandas	🔪 Chopping and preparing the ingredients
Train	scikit-learn, XGBoost	👩‍🍳 Trying several recipes
Track	**MLflow**	📒 A lab notebook where every attempt is written down
Gate + registry	**MLflow Model Registry**	🏅 The jury picks the dish that goes on the menu
API	**FastAPI**	🧑‍🍳 The waiter who takes orders and brings the dish
Container	**Docker**	🚚 A food-truck: the whole kitchen, ready to park anywhere
CI/CD	**GitHub Actions**	👮 The health inspector checks every change before opening
Dashboard	**Streamlit**	🪑 The dining room where customers see and enjoy the food
Monitor	KS drift test	📋 Checking whether customers still order the same things

📂 Where is each piece?

```
GoldMLFlow/
├── params.yaml              ← 🎛️ control panel: every setting lives here
├── dvc.yaml / dvc.lock      ← 🗃️ pipeline recipe + exact snapshot of every stage
├── src/
│   ├── data/extract.py      ← ⛏️ stage 1
│   ├── data/validate.py     ← 🔍 stage 3
│   ├── features/build.py    ← 🧩 stage 4
│   ├── models/train.py      ← 🧪 stage 5
│   ├── models/promote.py    ← 🚦🏆 stage 6
│   ├── api/main.py          ← 🍽️ stage 7
│   └── monitoring/drift.py  ← 📡 stage 10
├── app/dashboard.py         ← 📊 Streamlit app
├── notebooks/               ← 📓 exploration (one per stage) → became src/*.py
├── tests/                   ← ✅ automatic tests (run by CI)
├── Dockerfile(.dashboard), docker-compose.yml  ← 🐳
└── .github/workflows/       ← ⚙️ CI (ci.yml) and CD (cd.yml)
```

⚠️ Honest expectations

Hourly gold moves are **almost random**. Our best model is right **~51.5%** of the time (the "always UP" baseline gets ~50.9%). That is normal and is **not** the point: this project teaches the **process**. If a trading model ever shows 90% accuracy, suspect a bug (see data leakage).

> 📌 Educational project. **Not financial advice.**

---

01 · Setup & run everything

🍳 **Analogy**	Preparing the kitchen before cooking
🎯 **Goal**	Have Python, the libraries, MT5, Docker and Git ready
⏱️ **Time**	~15 minutes

✅ Requirements

Tool	Why	Check
Windows 10/11	The `MetaTrader5` Python package only exists for Windows	—
MetaTrader 5 terminal (open & logged in, demo account is fine)	Source of gold prices	open the app
Python 3.11 or 3.12	Runs everything	`python --version`
Git	Versions the code	`git --version`
Docker Desktop	Containers	`docker --version`
GitHub account	CI/CD	—

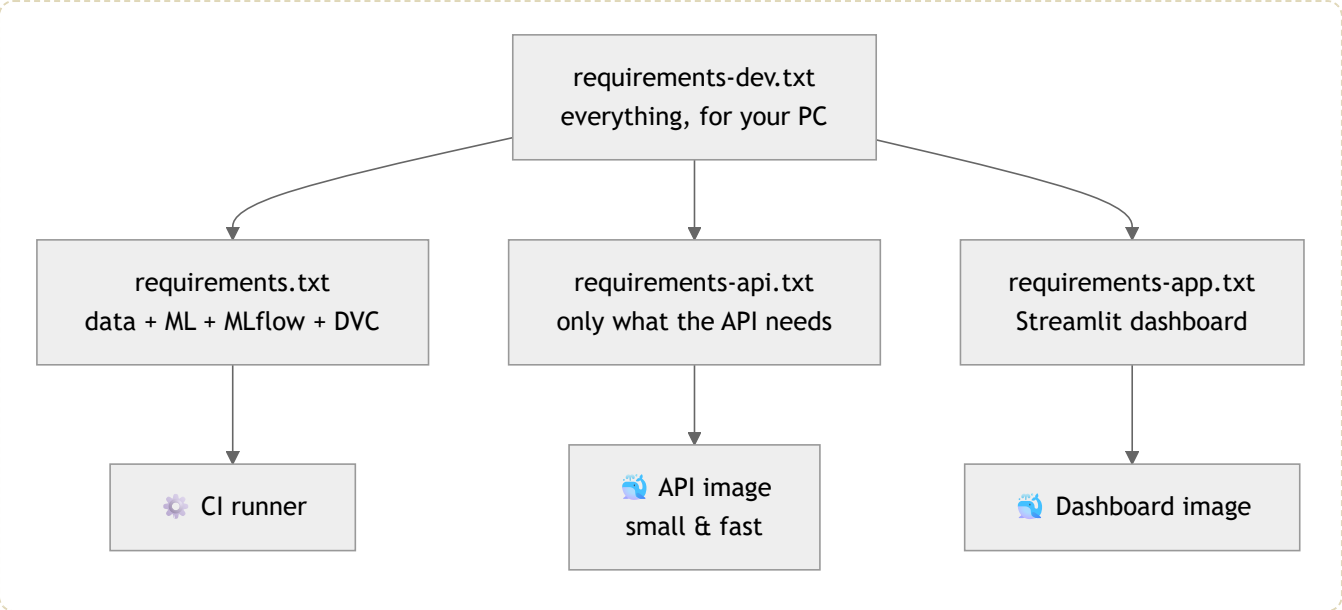
🧰 Step by step

```
# 1) Get the code
git clone https://github.com/MirandaCR/GoldMLFlow.git
cd GoldMLFlow

# 2) Create an isolated Python environment ("a clean kitchen just for this recipe")
python -m venv .venv
.\.venv\Scripts\Activate.ps1

# 3) Install everything (core + API + dashboard + notebooks + tests)
pip install -r requirements-dev.txt
```

📦 Why 4 requirements files?



Each container installs **only what it needs** → smaller, faster and safer images.

🚀 Run the whole pipeline (one command)

With MetaTrader 5 **open**:

```
dvc repro          # extract → validate → features → train → promote → drift
```

You'll see each stage run in order. Run it again and DVC says "*didn't change, skipping*": it only re-runs what changed.

👀 Look at the results

```
mlflow ui --backend-store-uri sqlite:///mlflow.db     # 🔦 experiments  → http://localhost:5000
uvicorn src.api.main:app                               # 🍽️ API          →
http://localhost:8000/docs
streamlit run app/dashboard.py                         # 📊 dashboard    → http://localhost:8501
docker compose up --build                              # 🐬 API + dashboard in containers
python scripts/live_demo.py                            # ✨ live MT5 prediction in the terminal
jupyter lab notebooks/                                 # 📓 the exploration notebooks
pytest                                                 # ✅ tests
```

> 💡 **Port 8000 busy?** (another app is using it) → `$env:API_PORT=8010; docker compose up` and use `http://localhost:8010`.

🧪 No MetaTrader 5? (Mac/Linux or no account)

Everything except extraction still works with the **sample** stored in Git (`data/sample/xauusd_h1_sample.csv` , last 4,000 candles):

```
python -m src.data.validate  --input data/sample/xauusd_h1_sample.csv
python -m src.features.build --input data/sample/xauusd_h1_sample.csv --output
data/processed/features.csv
python -m src.models.train
python -m src.models.promote
```

That's exactly what the CI robot does on GitHub (Linux, no MT5).

---

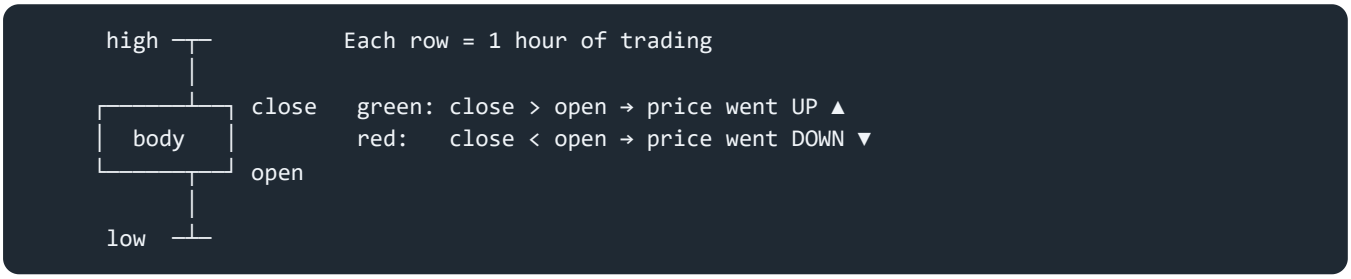
02 · Extract gold data from MetaTrader 5

🍳 **Analogy**	🛒 Going to the market to buy the ingredients
🎯 **Goal**	Download hourly (H1) gold candles into a CSV
📓 **Notebook**	`notebooks/01_extract_mt5.ipynb`
🏭 **Production code**	`src/data/extract.py`
▶️ **Run**	`dvc repro extract` (or `python -m src.data.extract --sample`)

🔌 How it works



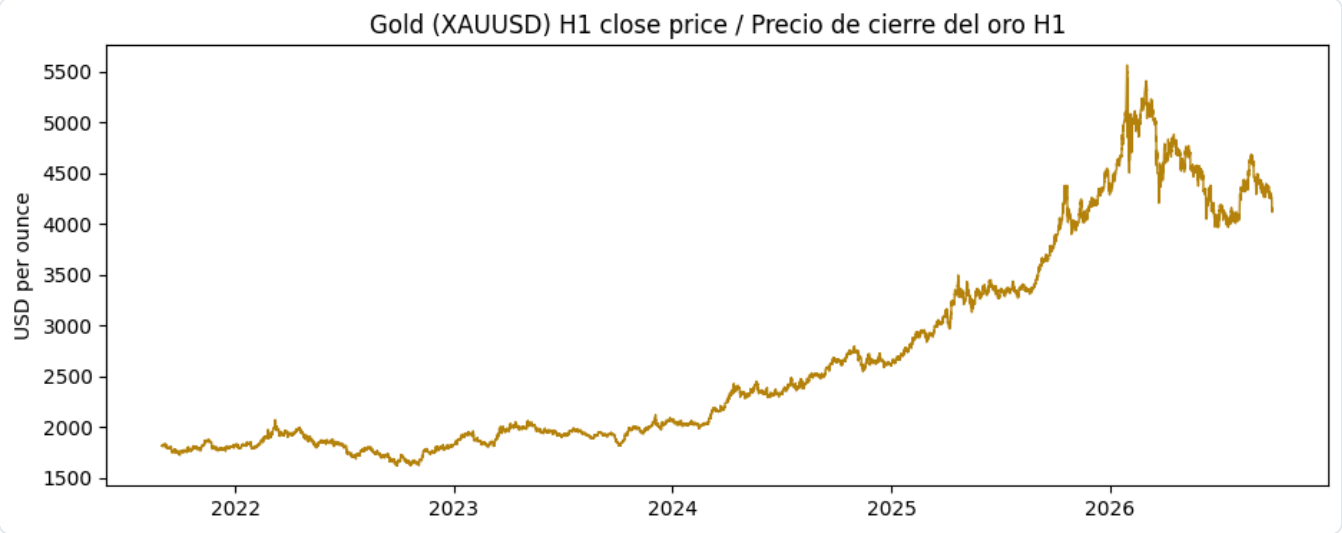
🕯️ What is a candle?



time	open	high	low	close	volume
2021-08-31 18:00	1807.03	1808.94	1804.15	1807.78	9978
2021-08-31 19:00	1807.86	1814.04	1807.42	1813.83	5287

📊 How much history is there?

Our broker calls gold `GOLD` and has **82,525 H1 candles (June 2001 → today)**. We use the **last 30,000 (~5 years: Aug 2021 → Sep 2026)**.



Why not all 25 years? Markets change: gold in 2005 had different volatility, liquidity and trading hours. Five years give ~30k examples and stay relevant. It's a setting (`params.yaml → data.n_bars`), so you can try more.

🧠 Key details that matter in production

Detail	Why
Symbol auto-detection (`symbol_candidates`)	Every broker names gold differently
Drop the last candle	It's still forming; its "close" will change → it would be a lie
Broker server time	MT5 timestamps are in the broker's time zone (usually GMT+2/+3)
Import MT5 inside the function	So the rest of the project works on Linux (CI/Docker) where MT5 doesn't exist
Sample file (`--sample`)	A small copy (4,000 candles) stored in Git for CI tests

🧪 Try it yourself

1. In `params.yaml`, change `n_bars: 30000` to `50000` and run `dvc repro`. What does DVC re-run? (Hint: *everything*, because the raw data changed.)
2. Change `timeframe: H1` to `H4`. How many candles do you get? How does accuracy change?

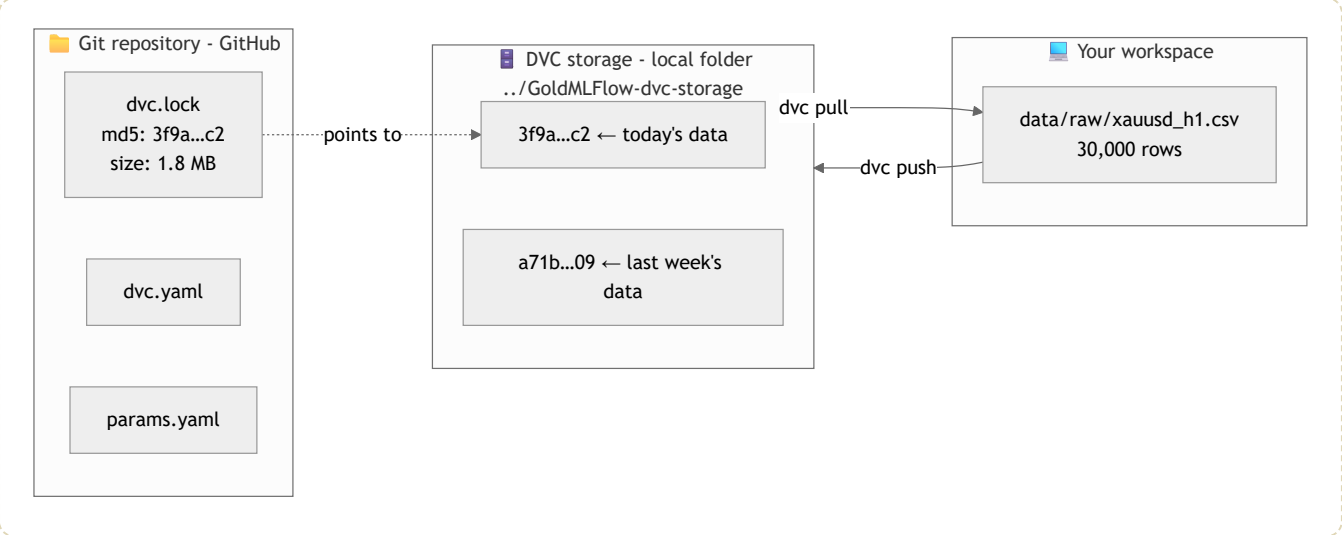
---

03 · Data Version Control (DVC)

🔍 **Analogy**	📸 Dated photos of your pantry: you can always go back to yesterday's
🎯 **Goal**	Version big data files + make the pipeline reproducible
🏭 **Files**	`dvc.yaml`, `dvc.lock`, `.dvc/config`, `params.yaml`
▶️ **Run**	`dvc repro`, `dvc push`, `dvc pull`, `dvc dag`, `dvc metrics show`

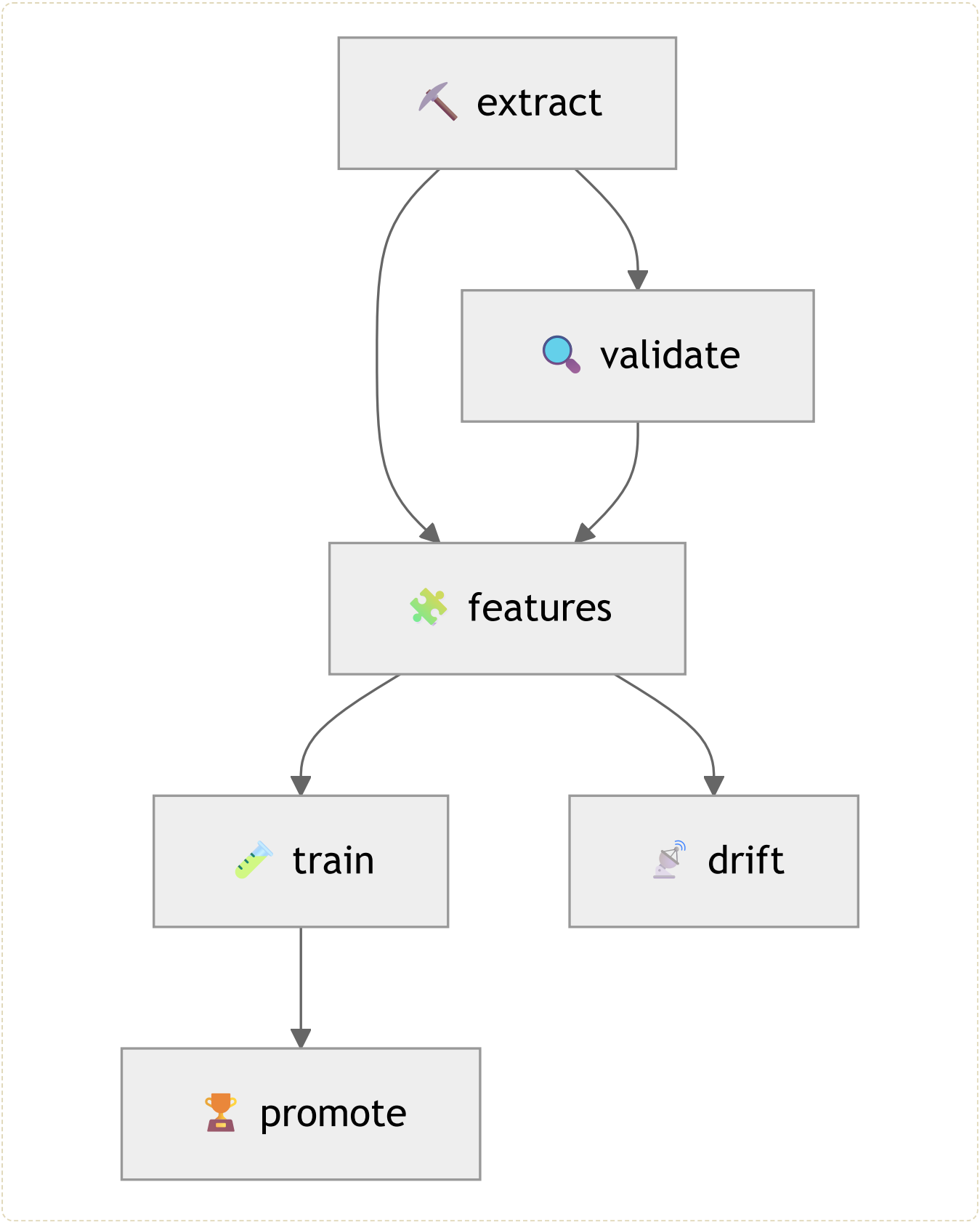
🤔 Why not just put the CSV in Git?

Git is great for **code** (small text files), bad for **data** (big files that change entirely). DVC = **"Git for data"**: Git stores a tiny *fingerprint* (hash) of the file; the file itself lives in a **storage** (here, a local folder).



🧬 Our pipeline (`dvc.yaml`)

Each **stage** declares: the command, what it **depends on** (code, data, params) and what it **produces**.



(This is what `dvc dag` prints.)

```
train:
  cmd: python -m src.models.train
  deps:                          # if any of these change → re-run
    - src/models/train.py
    - data/processed/features.csv
  params:
    - train                      # only the "train:" section of params.yaml
  outs:
    - reports/train_results.json
```

🧠 The magic: `dvc repro` only re-runs what changed

You change...	DVC re-runs...
`train.models.xgboost.max_depth` in params.yaml	train → promote
`features.rsi_window`	features → train → promote → drift
`src/monitoring/drift.py`	only drift
nothing	nothing (*"didn't change, skipping"*)

📸 `dvc.lock` = the exact snapshot

After each run, `dvc.lock` records the **md5 of every input and output**. Commit it with Git → anyone can reproduce **exactly** the same result months later:

```
git checkout <old-commit>   # code + dvc.lock of that day
dvc pull                    # the data of that day
dvc repro                   # same results
```

🗄️ Our local storage

```
dvc remote add -d localstorage ../GoldMLFlow-dvc-storage   # already configured in .dvc/config
dvc push      # upload data versions to the storage
dvc pull      # download the data that matches the current dvc.lock
```

> 🏢 In a company, the storage would be S3, Google Cloud Storage, Azure Blob or a shared drive: just change the remote URL. Everything else stays the same.

🔀 What goes to Git vs DVC?

File	Where	Why
`data/raw/xauusd_h1.csv` (1.8 MB, changes)	**DVC**	big data
`data/processed/features.csv`	**DVC**	generated data
`data/sample/xauusd_h1_sample.csv` (small)	**Git** (`cache: false`)	CI needs it and can't reach the local storage
`models/champion/` (371 KB)	**Git** (`cache: false`)	the Docker image is built from it in CD
`reports/metrics.json`	**Git** (`metrics:`)	compare runs with `dvc metrics diff`

🧪 Try it yourself

```
dvc dag                       # draw the pipeline
dvc metrics show              # current metrics
# change train.models.xgboost.max_depth to 5 in params.yaml, then:
dvc repro
dvc metrics diff              # compare with the last commit
dvc params diff               # which params changed
```

---

04 · Validate data & build features

🍳 **Analogy**	🔍 Checking the ingredients aren't rotten, then 🔪 chopping them
🎯 **Goal**	Guarantee healthy data, then turn candles into numeric "clues" + the answer
📓 **Notebooks**	`02_validate_explore`, `03_features`
🏭 **Production code**	`src/data/validate.py`, `src/features/build.py`
✅ **Tests**	`tests/test_data.py`, `tests/test_features.py`

Part A · 🔍 Validation — "garbage in, garbage out"

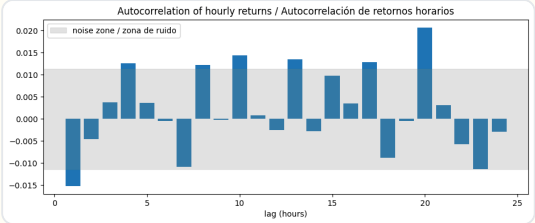
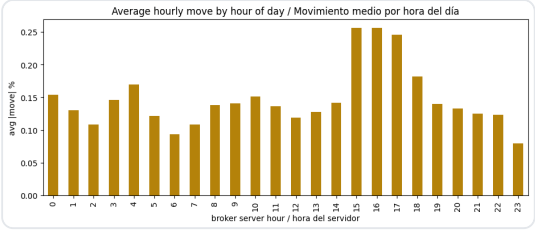
Before any model sees the data, 8 yes/no questions:

Check	Catches...
required_columns	a broken export
enough_rows (≥ 1,000)	MT5 returned almost nothing
no_missing_values	empty cells
no_duplicated_timestamps	the same hour twice
time_is_sorted	shuffled rows
prices_positive	zeros / negative prices
high_is_highest / low_is_lowest	impossible candles

If **any** check fails → the script exits with code 1 → **DVC stops the pipeline**. No model is trained on broken data.

```
✅ required_columns    ✅ enough_rows    ✅ no_missing_values    ✅ no_duplicated_timestamps
✅ time_is_sorted      ✅ prices_positive    ✅ high_is_highest    ✅ low_is_lowest
Data validation passed (29999 rows)
```

🔎 What exploring the data taught us

Finding	Chart	Consequence
UP 50.9% / DOWN 49.1%	—	"Always UP" is already right 50.9% → that's the **baseline to beat**
Hourly returns barely correlate with the past		Expect ~51–53%, not 90%
Gold moves most at London / New York opens		Give the model the **hour of day**

Part B · 🧩 Features — turning candles into clues

A model can't read a chart. For each candle *t* we compute numbers that describe **only the past**, plus the **target**:

```
candle t-200 … t-1, t   ──▶   clues(t)   ──▶ 🤖 ──▶  will candle t+1 close higher?
      (only the PAST)                                  target = 1 (UP) / 0 (DOWN)
```

Family	Clues	Plain words
🤸 Momentum	`ret_1h, ret_2h, ret_3h, ret_5h, ret_10h, ret_24h`	How much did price move in the last N hours?
📈 Trend	`dist_sma_10, dist_sma_50, dist_sma_200`	Is price above or below its average?
🎢 Oscillators	`rsi`, `macd_hist`	Is it "tired" of going up/down?
🍸 Risk	`volatility`, `range_pct`, `body_pct`	How nervous is the market?
📊 Activity	`volume_z`	Is there unusual activity?
🕐 Calendar	`hour_sin, hour_cos, dayofweek`	Asia? London? New York? Friday?

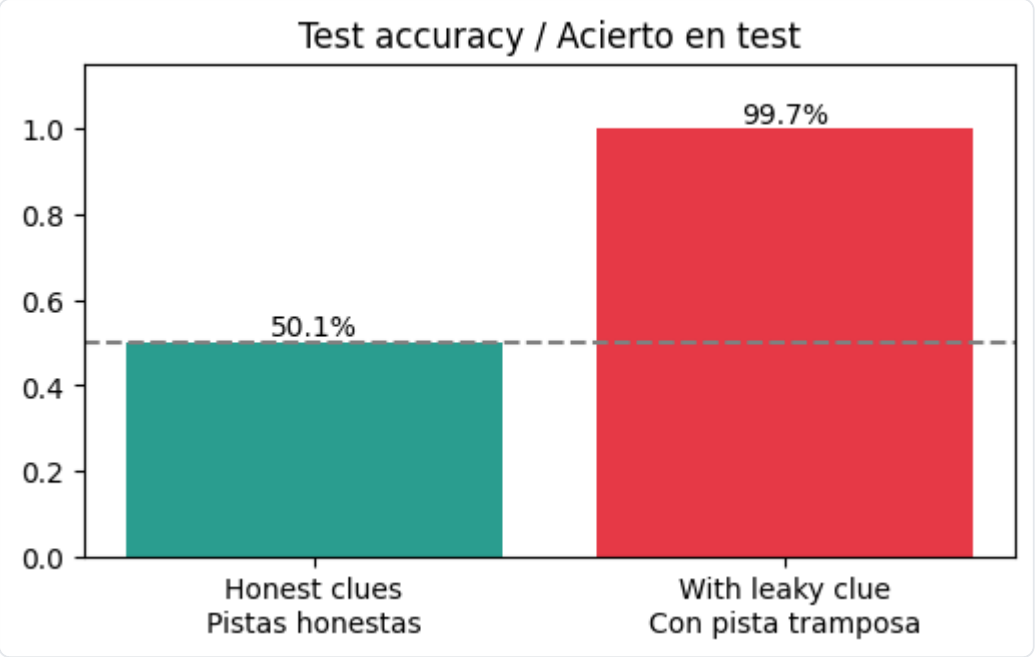
Result: 29,799 rows × 18 clues + target.

> 🕐 **Why sin/cos for the hour?** To a model, 23:00 and 00:00 look far apart (23 vs 0), but they're neighbours. Placing hours on a circle fixes it.

🚨 Data leakage: the #1 trap

Leakage = a clue that accidentally contains information from the future. On paper the model looks like a genius; in real life it loses money, because in real time the future doesn't exist yet.

In notebook 03 we add one innocent-looking clue: a moving average **centered** on the candle (it averages the previous hour, the current hour… **and the next hour**):



50.1% → **99.7%** with a single line of code. **If results look too good to be true, they are.**

🛡️ How production protects us

```
def test_no_leakage_features_ignore_the_future(sample_raw, params):
    """Changing FUTURE prices must not change today's clues."""
    base = make_features(sample_raw, ...)
    tampered = sample_raw.copy()
    tampered.loc[last 100 rows, prices] *= 1.5       # 🔨 change the future
    after = make_features(tampered, ...)
    assert clues of the older rows are identical      # ✅ or CI fails
```

This test runs on **every push** in GitHub Actions.

♻️ Same code for training and serving

```
make_features(df, params, with_target=True)   # training: adds target, drops last candle (no
future yet)
make_features(df, params, with_target=False)  # API: same clues, keeps the last candle to predict
it
```

Using **one function** in both places avoids *training/serving skew* (the API computing clues slightly differently from training → silent errors).

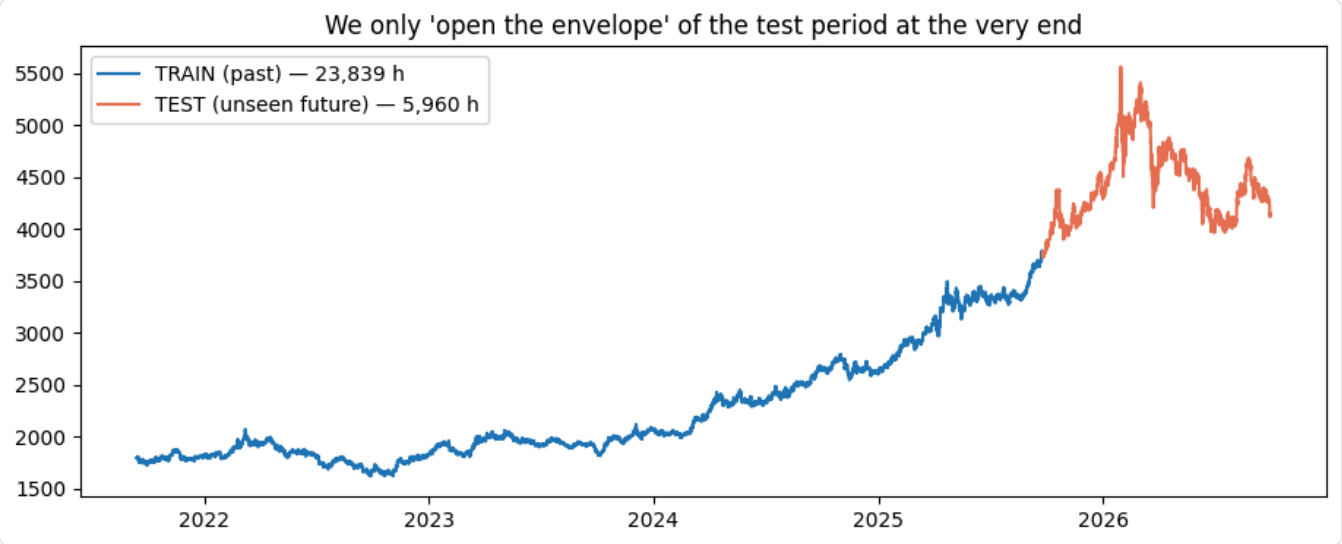
---

05 · Train models & track them with MLflow

🍳 **Analogy**	👩‍🍳 Trying several recipes and 📒 writing everything down in a lab notebook
🎯 **Goal**	Compare models fairly and never lose a result
📓 **Notebook**	`04_train_mlflow`
🏭 **Production code**	`src/models/train.py`, `src/models/tracking.py`
▶️ **Run**	`dvc repro train` · UI: `mlflow ui --backend-store-uri sqlite:///mlflow.db`

⏳ Rule #1 for time series: never shuffle

```
|■■■■■■■■■■■■■■■■■■■■■■ TRAIN 80% (past) ■■■■■■■■■■■■■■■■■■■■■■|□□□ TEST 20% (unseen future) □□□|
 2021-09-13 ──────────────────────────────────────────────── 2025-09-24 ──────────────────── 2026-09-29
            23,839 hours                                              5,960 hours
```



Inside the training part we use **walk-forward cross-validation** (`TimeSeriesSplit`): "practice exams", always training on the past and testing on what comes next.

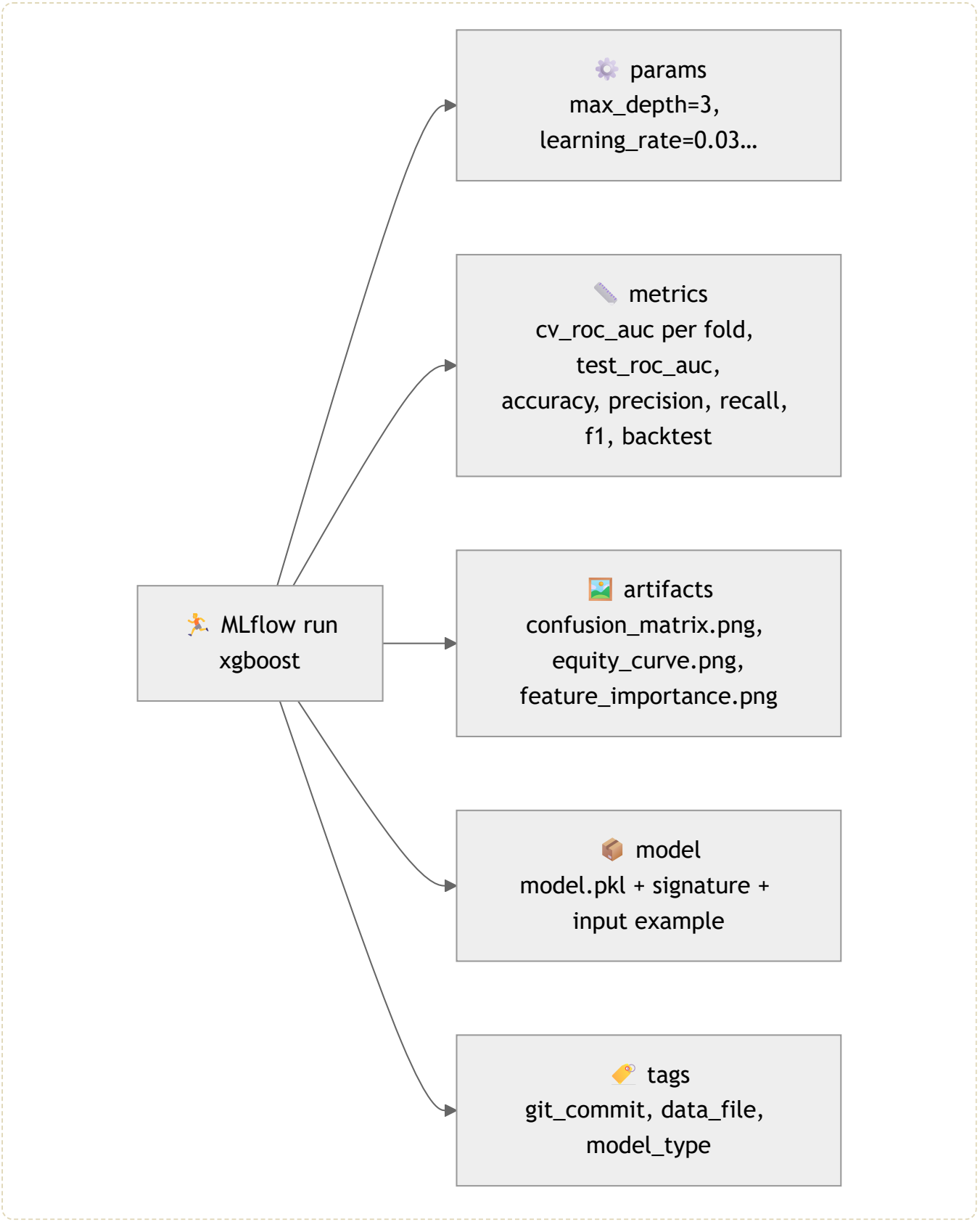
```
fold 1: ▓▓▓▒
fold 2: ▓▓▓▓▒
fold 3: ▓▓▓▓▓▒          ▓ = train   ▒ = validate
fold 4: ▓▓▓▓▓▓▒
fold 5: ▓▓▓▓▓▓▓▒
```

🤖 The 4 contestants

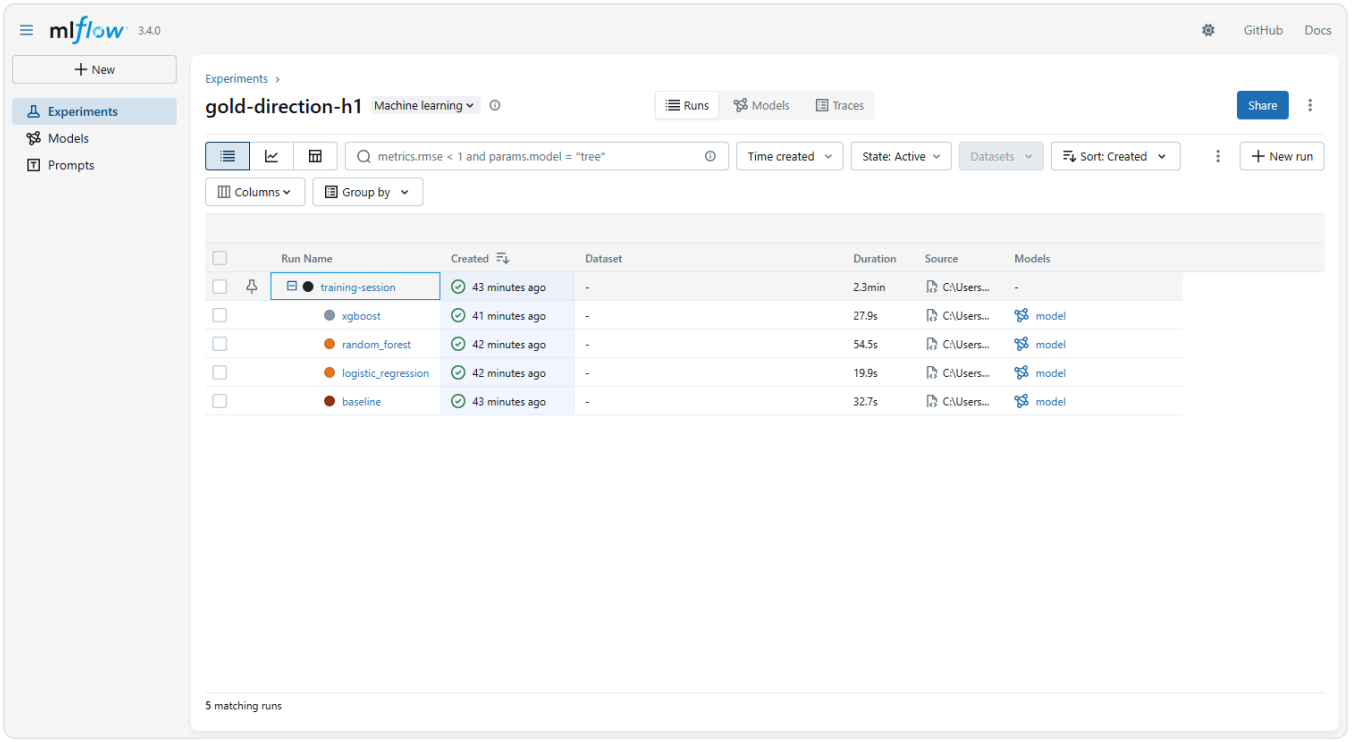
Model	Idea	Why it's here
baseline (`DummyClassifier`)	always says the most common answer (UP)	🪙 the "coin flip" every model must beat
logistic_regression	a weighted sum of the clues	simple, fast, hard to overfit
random_forest	300 decision trees vote	captures non-linear patterns
xgboost	trees that fix each other's mistakes	usually the strongest on tabular data

Every model is an sklearn `Pipeline` (e.g. scaler + model) so preprocessing **travels with the model** to the API.

📒 What MLflow records for every model

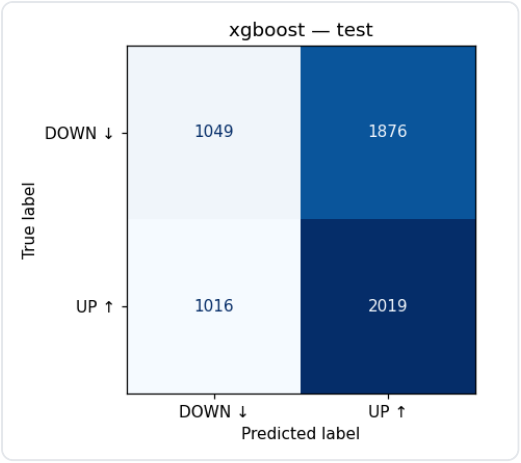
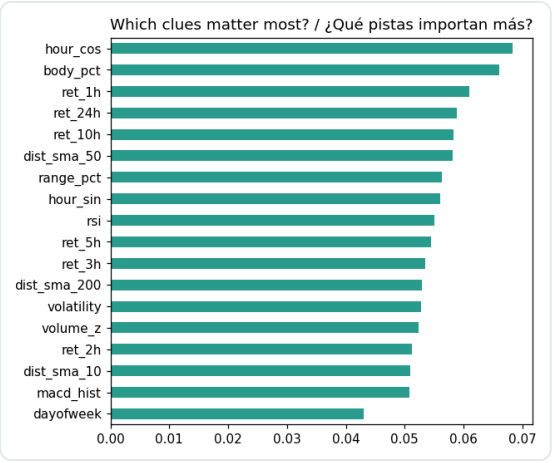


Runs are **nested**: one parent `training-session` with one child per model.



🖼️ Charts saved inside each run



Confusion matrix	Feature importance
	

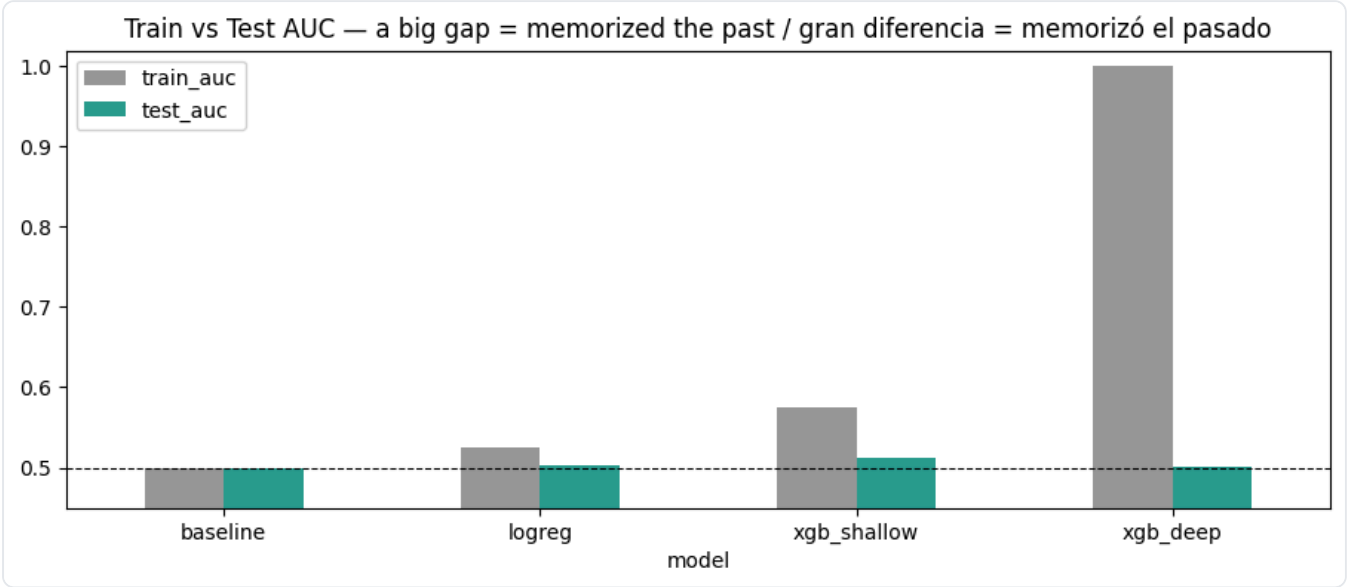
✍️ The "signature": the model's contract

MLflow stores which columns (and types) go in and what comes out. If someone later sends the wrong columns, MLflow warns instead of silently predicting garbage.

📊 Results (test = the last 12 months, never seen)

Model	test ROC-AUC	test accuracy
🥇 **xgboost**	**0.5140**	**51.48%**
random_forest	0.5032	50.45%
logistic_regression	0.5028	50.45%
baseline 🪙	0.5000	50.92%

🎓 Lesson from the notebook: overfitting



`xgb_deep` (8 levels deep) scores **0.999 AUC on the past** but **0.501 on the future**: it **memorized noise**. On noisy markets, simpler models generalize better.

🗂️ Where is the data stored?

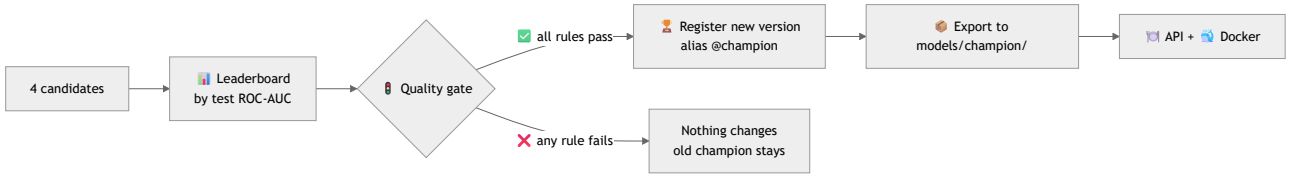
Mode	How	Use
Local (default)	`mlflow.db` (SQLite) + `mlartifacts/` folder	learning, one person
Team	`docker compose --profile team up mlflow` + `$env:MLFLOW_TRACKING_URI="http://localhost:5000"`	a shared server for everyone

---

06 · Evaluate, quality gate & Model Registry

🔍 **Analogy**	🏅 The jury tastes every dish; only the best one goes on the menu
🎯 **Goal**	Promote a model **only** if it's objectively better than guessing
📓 **Notebook**	`05_evaluate_registry`
🏭 **Production code**	`src/models/promote.py` , `gate.py` , `check_gate.py`

🚦 The flow



📏 Metrics in plain words

Metric	Question it answers	Our champion
Accuracy	% of hours where UP/DOWN was right	51.48%
Precision	When it says UP, how often is it really UP?	51.84%
Recall	Of all real UPs, how many did it catch?	66.52%
ROC-AUC	Does it rank "more likely UP" above "less likely UP"? 0.5 = coin	0.5140

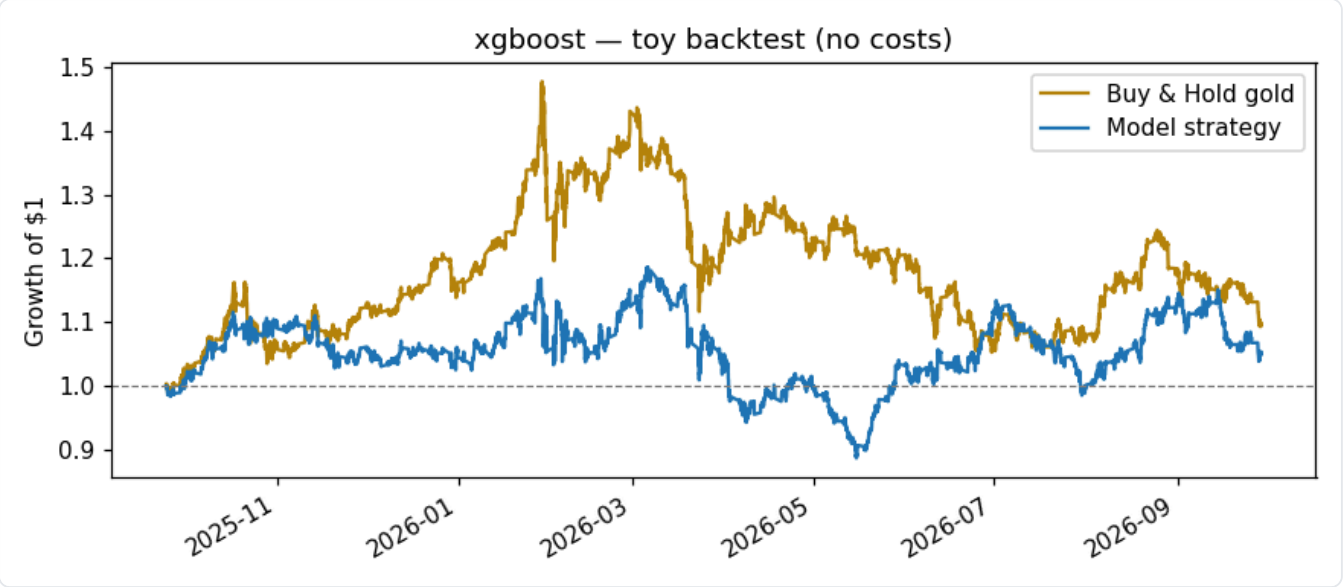
🚦 The quality gate — written rules, not opinions

Defined in `params.yaml → evaluate` :

```
evaluate:
  primary_metric: roc_auc
  min_roc_auc: 0.51          # must be better than a coin by a minimum margin
  must_beat_baseline: true   # must beat "always UP" in AUC AND accuracy
```

```
Leaderboard (test_roc_auc):
  1. xgboost              test_roc_auc=0.5140  acc=0.5148
  2. random_forest        test_roc_auc=0.5032  acc=0.5045
  3. logistic_regression  test_roc_auc=0.5028  acc=0.5045
  4. baseline             test_roc_auc=0.5000  acc=0.5092
  ✅ roc_auc >= 0.51
  ✅ beats baseline (roc_auc)
  ✅ beats baseline (accuracy)
🏆 Registered gold-direction-h1 v2 → alias @champion
```

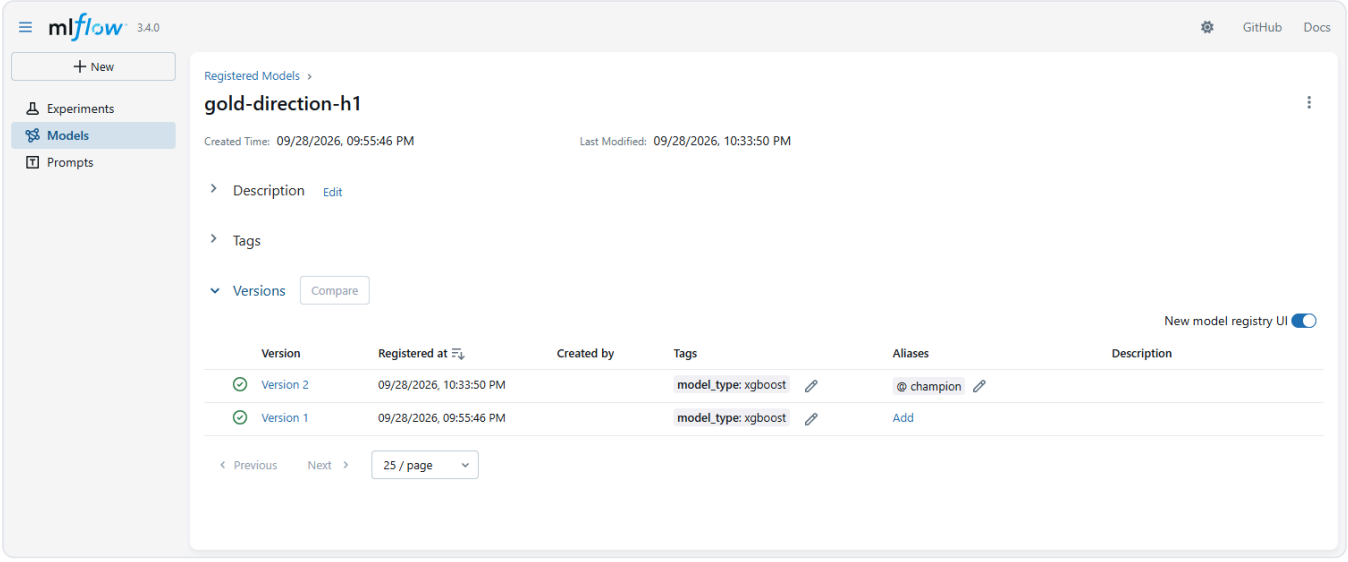
💰 The honest truth: beating 50% ≠ making money



Toy strategy (buy if UP, sell if DOWN, **no fees**): **+5.3%**. Simply holding gold: **+9.8%**. The model is *slightly* better than a coin at predicting direction, but that doesn't mean it's a good trading system. This project is about the **process**.

🏆 The Model Registry

Like a library catalog: every approved model gets a **version number**; an **alias** (`@champion`) points to the one in production.




```
# Load the production model BY NAME — no file paths:
model = mlflow.sklearn.load_model("models:/gold-direction-h1@champion")
```

Action	How
Promote	move `@champion` to the new version (done automatically by `promote.py`)
Rollback	move `@champion` back to an older version (1 click in the UI)
Audit	every version links to its run: params, metrics, charts, git commit

📦 Why also export to `models/champion/`?

The GitHub robot (CD) builds the Docker image but **can't reach your local MLflow**. So the champion is also copied into the repo (371 KB): `model/` (MLflow format) + `metadata.json` (metrics, features, version).

🛡️ The gate also runs in CI

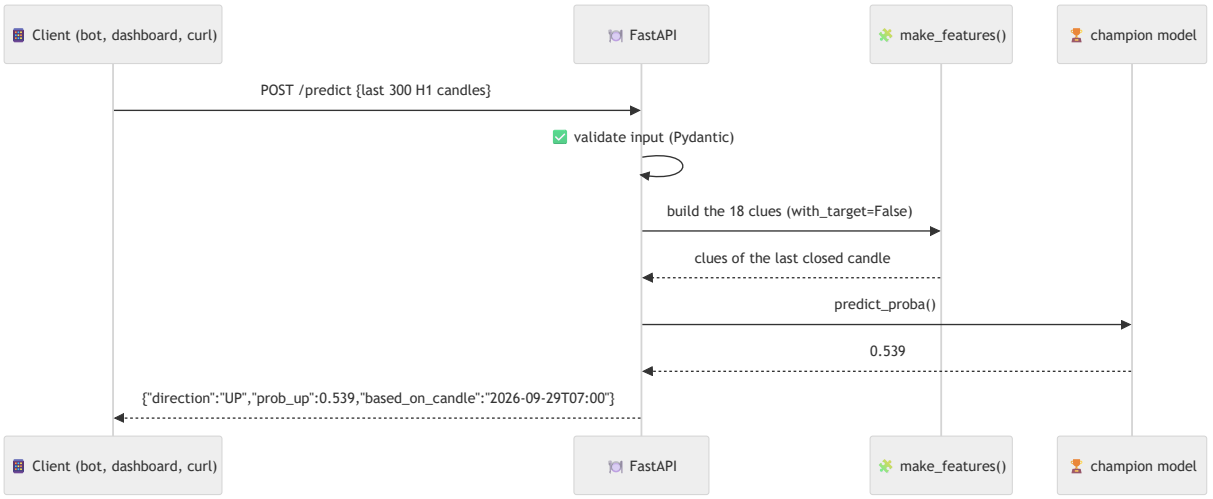
`python -m src.models.check_gate` runs on **every push**: if someone commits a champion that doesn't pass the rules, the build turns ❌ red and can't be merged.

---

07 · Model API (FastAPI)

🍳 **Analogy**	🧑‍🍳 The waiter: takes orders (requests) and brings back the dish (prediction)
🎯 **Goal**	Let **any** program ask "UP or DOWN?" over HTTP
📓 **Notebook**	`06_serve_api`
🏭 **Production code**	`src/api/main.py`, `src/api/schemas.py`
▶️ **Run**	`uvicorn src.api.main:app` → http://localhost:8000/docs

🔄 How a prediction travels

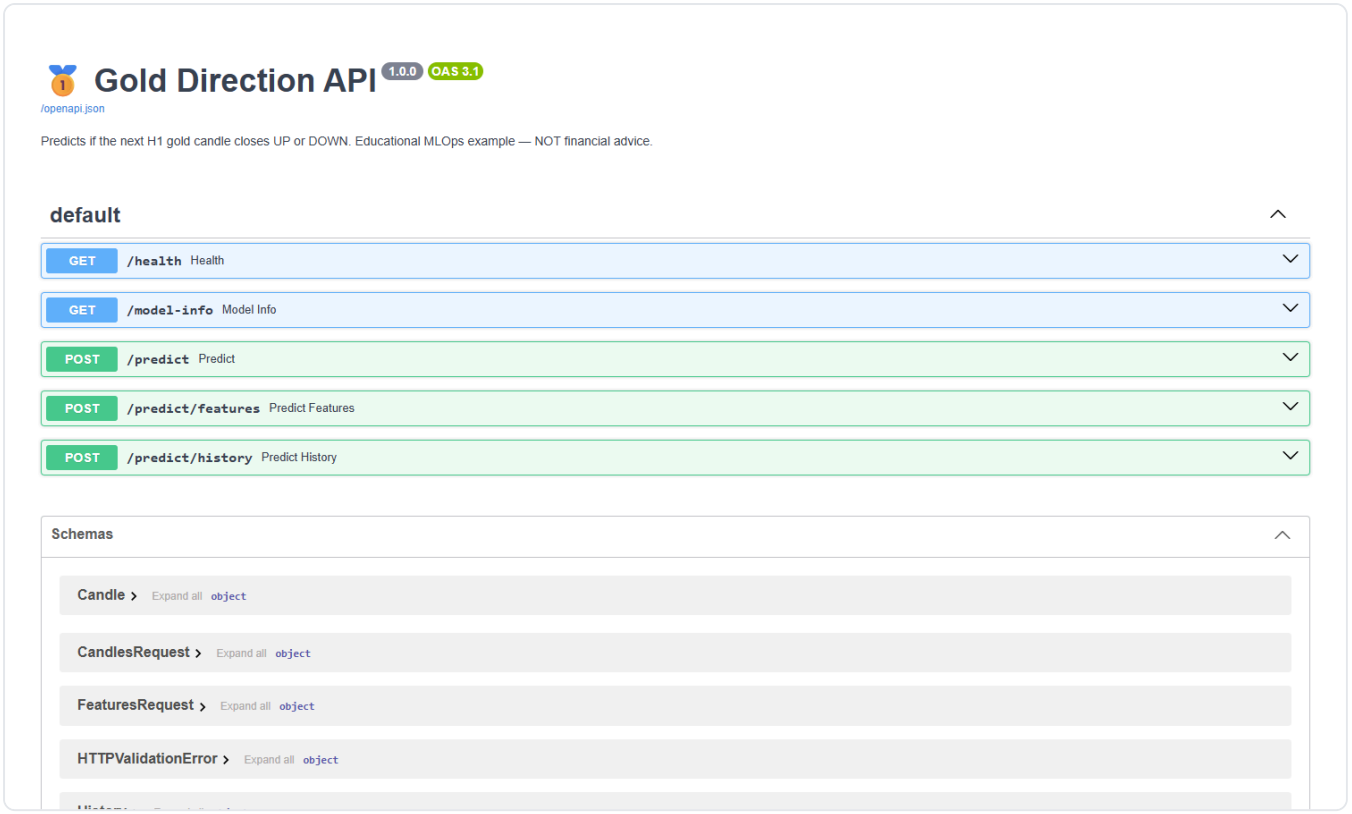


🚪 Endpoints

Method	Path	What for
GET	`/health`	"Are you alive?" (used by Docker healthcheck)
GET	`/model-info`	Which model, version, metrics, features
POST	`/predict`	Send ≥ 250 candles → UP/DOWN for the **next** hour
POST	`/predict/features`	Send the 18 clues directly (advanced)
POST	`/predict/history`	P(UP) for **every** candle sent (used by the dashboard charts)

📖 Free interactive documentation (Swagger)

FastAPI builds this page automatically from the code. Click **"Try it out"** to call the API from the browser:



💻 Calling it

```
# PowerShell
Invoke-RestMethod http://localhost:8000/health
Invoke-RestMethod http://localhost:8000/model-info
```

```
# Python
import httpx, pandas as pd
candles = pd.read_csv("data/sample/xauusd_h1_sample.csv").tail(300)
r = httpx.post("http://localhost:8000/predict", json={"candles":
candles.to_dict(orient="records")})
print(r.json())   # {'direction': 'DOWN', 'prob_up': 0.452, ...}
```

```
# 🔮 The full live loop: MT5 → API → prediction
python scripts/live_demo.py
```

```
╔══════════════════════════════════════════════╗
  🥇 GOLD H1 — next candle prediction
  Last closed candle : 2026-09-29T07:00:00   close=4126.53
  Model              : xgboost v2  (test AUC 0.514)
  Prediction         : 🟢 ▲ UP / SUBE
  Probability UP     : 53.9%
╚══════════════════════════════════════════════╝
```

🛡️ Good practices inside

Practice	Where
Input validation (Pydantic): ≥ 250 candles, prices > 0 → clear 422 error, never a crash	`schemas.py`
Model loaded **once** at startup, not on every request	`lifespan()`
Model location configurable (`MODEL_DIR` env var)	`main.py`
Same feature code as training (no training/serving skew)	`make_features(with_target=False)`
Tests with an in-memory client (`TestClient`) on every push	`tests/test_api.py`

---

08 · Containers (Docker)

🍳 **Analogy**	🚚 A food-truck: the whole kitchen (Python, libraries, code, model) packed so it can park anywhere
🎯 **Goal**	Kill "it works on my machine": the same box runs on your PC, in CI and in the cloud
🏭 **Files**	`Dockerfile`, `Dockerfile.dashboard`, `docker-compose.yml`, `.dockerignore`
▶️ **Run**	`docker compose up --build`

📦 Image vs container

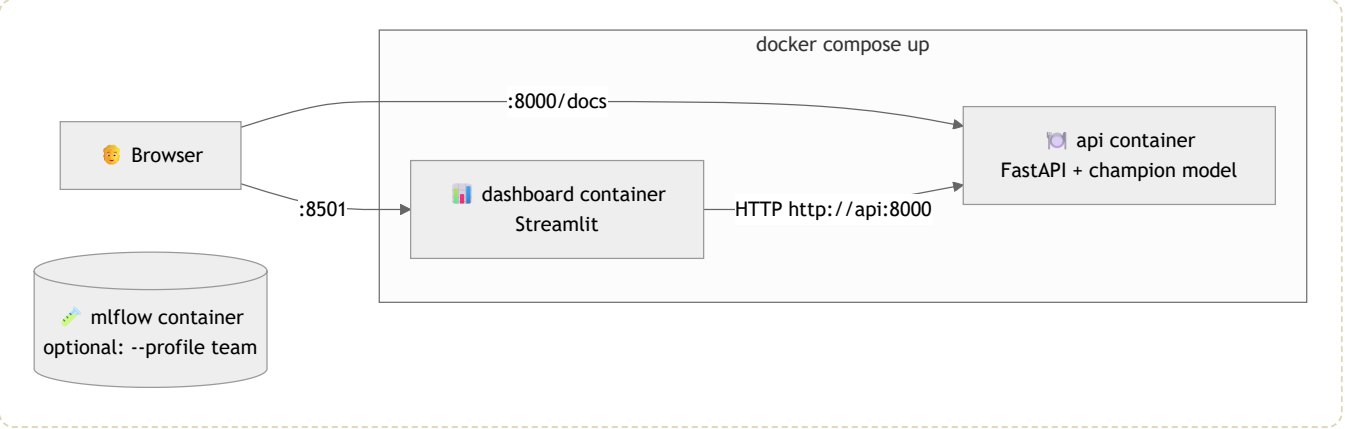
Concept	Analogy	Command
Dockerfile	the recipe to build the truck	—
Image	the finished truck, parked (read-only)	`docker build`
Container	the truck **open and serving**	`docker run`
Registry (ghcr.io)	the parking lot where trucks are shared	`docker push / pull`
Compose	a fleet of trucks that work together	`docker compose up`

🧱 Our API image, layer by layer

```
FROM python:3.12-slim                          # 1. a small Linux + Python
COPY requirements-api.txt .
RUN pip install -r requirements-api.txt        # 2. libraries (cached layer → fast rebuilds)
COPY src/ ... models/champion/ ...             # 3. only the code the API needs + the model
RUN useradd appuser && USER appuser            # 4. don't run as root (security)
HEALTHCHECK ... /health                        # 5. Docker checks it's alive every 30 s
CMD ["uvicorn", "src.api.main:app", ...]       # 6. what runs when the container starts
```

> 💡 **Why is XGBoost `xgboost-cpu` in Linux?** The normal package downloads ~350 MB of NVIDIA GPU libraries we don't use. `xgboost-cpu` keeps images small.

🚢 Two containers working together



- Containers find each other **by service name** (`http://api:8000`).
- The dashboard **waits** until the API is healthy (`depends_on: condition: service_healthy`).
- The dashboard never loads the model itself: it asks the API, like any real client.

```
docker compose up --build        # build + start both
docker compose ps                # status (api should say "healthy")
docker compose logs -f api       # live logs
docker compose down              # stop and remove
```

> ⚠️ **Port busy?** `$env:API_PORT=8010; $env:DASHBOARD_PORT=8502; docker compose up`

✅ Proof that it's reproducible

The same 300 candles sent to the API **on Windows** and **inside the Linux container** give exactly the same answer:

```
local (Windows) : {'direction': 'DOWN', 'prob_up': 0.452, 'based_on_candle': '2026-09-
29T05:00:00'}
docker (Linux)  : {'direction': 'DOWN', 'prob_up': 0.452, 'based_on_candle': '2026-09-
29T05:00:00'}
```

🟡 Why isn't MetaTrader 5 in a container?

The `MetaTrader5` package only works on **Windows with the terminal open**; Docker images are Linux. That's why the **extraction runs on your PC** and everything downstream uses the data versioned by DVC. This separation is common in real life (data sources that only live in one place).

---

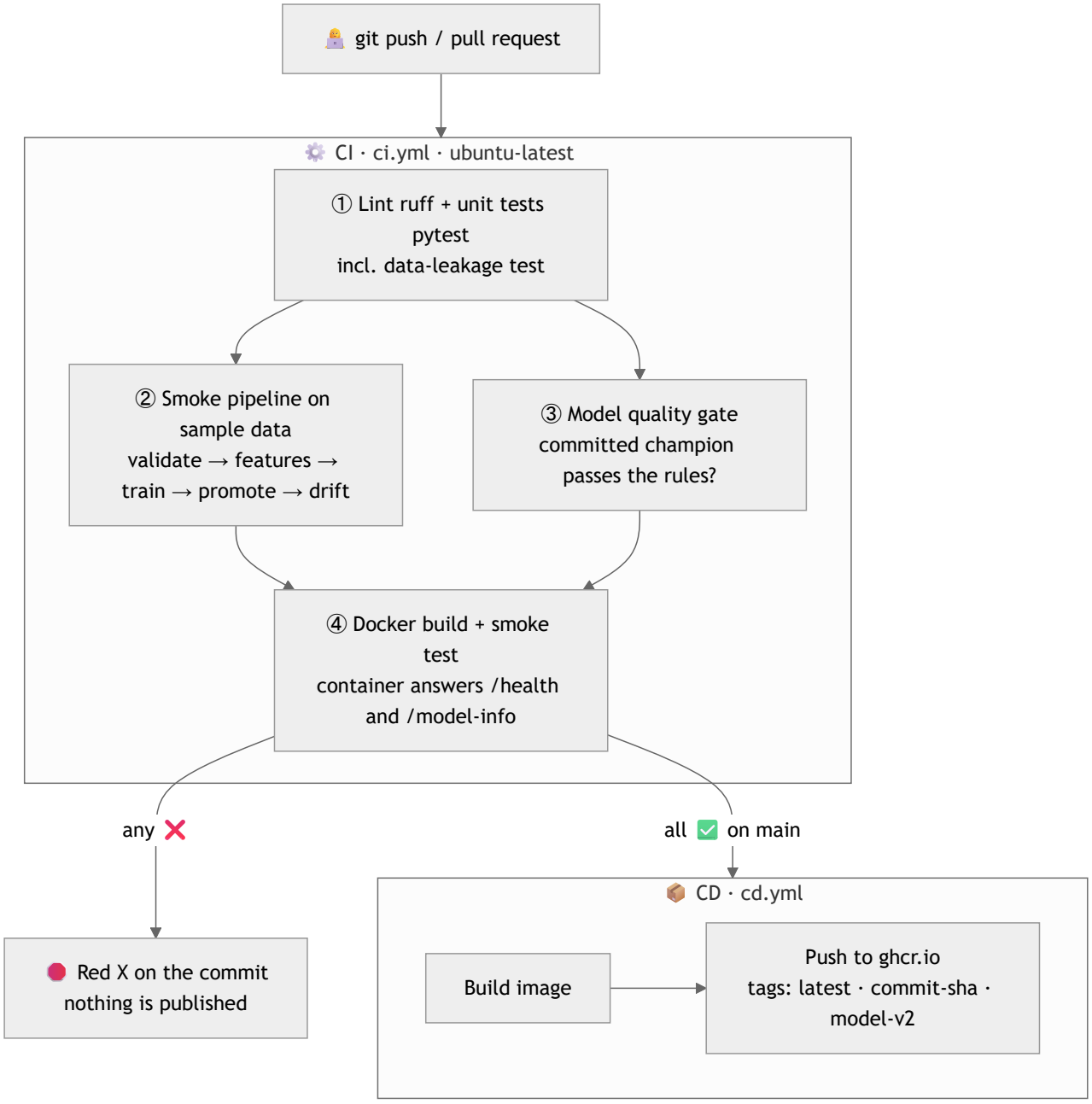
09 · CI/CD with GitHub Actions

🔍 **Analogy**	👮 The health inspector checks every change **before** the restaurant opens with it; then 🚚 the new truck is delivered automatically
🎯 **Goal**	Every push is tested automatically; only green changes are published
🏭 **Files**	`.github/workflows/ci.yml`, `.github/workflows/cd.yml`
🌐 **See it**	https://github.com/MirandaCR/GoldMLFlow/actions

🤔 CI vs CD

	CI — Continuous Integration	**CD — Continuous Delivery**
Question	"Is this change safe?"	"Ship it!"
When	every push and pull request	only when CI is ✅ on `main`
Does	lint, tests, pipeline, quality gate, Docker build	builds & publishes the Docker image
Result	✅ / ❌ next to each commit	`ghcr.io/mirandacr/goldmlflow:latest`

The full robot flow



🔎 Each CI job in plain words

Job	What it catches
① **Lint & unit tests**	messy code, broken functions, **data leakage**, API errors
② **Smoke pipeline**	"the whole pipeline still runs end to end" (on the 4,000-candle sample in Git: the robot has no MT5 and can't reach your local DVC storage)
③ **Quality gate**	someone committed a **worse model** than the rules allow
④ **Docker build**	the image doesn't build or the container doesn't answer

Results appear in the job **Summary** (metrics table + champion checks) and artifacts (reports, MLflow runs) can be downloaded.

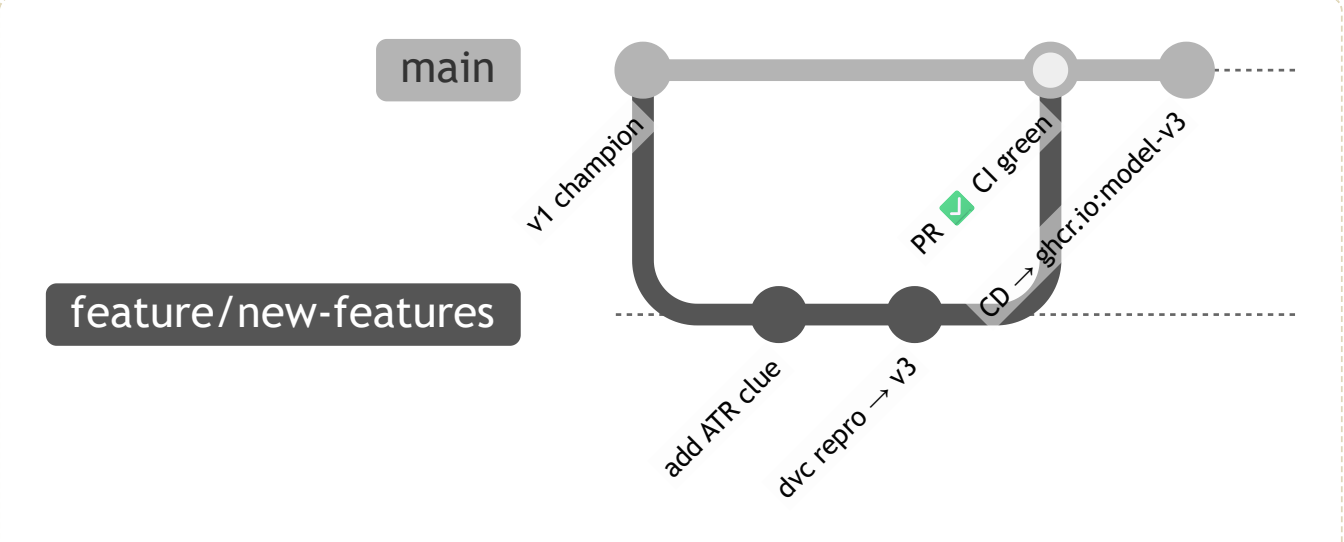
📦 CD: publishing the image

When CI is green on `main`, `cd.yml`: 1. reads the champion version from `models/champion/metadata.json` 2. builds the image and pushes it to **GitHub Container Registry** with 3 tags: `latest`, the commit SHA and `model-v2` 3. anyone can now run it without cloning anything:

```
docker run -p 8000:8000 ghcr.io/mirandacr/goldmlflow:latest
```

> 🔐 No secrets needed: the built-in `GITHUB_TOKEN` has `packages: write` permission (declared in the workflow). The first time, the package is **private**; make it public in GitHub → your profile → Packages → goldmlflow → Package settings.

🧭 The daily workflow with CI/CD



1. Create a branch, change code/params, `dvc repro`, `dvc push`.

2. `git push` → open a Pull Request → CI runs.
3. ✅ green → merge → CD publishes the new image automatically.

💡 Going further (real-world upgrades)

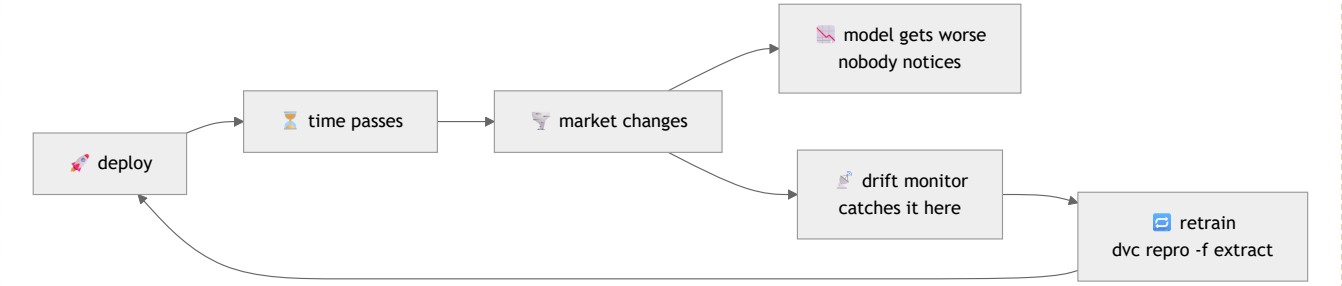
Upgrade	How
CI trains on the **full** data	use a cloud DVC remote (S3/GCS) + `dvc pull` in CI with secrets
Retrain on a schedule	add `on: schedule: - cron: "0 6 * * 1"`
Comment metrics on the PR	CML (`cml comment create`)
Deploy to a server	after the push, `ssh` / Cloud Run / Kubernetes step

---

10 · Monitoring & data drift

🔍 **Analogy**	📋 Checking whether customers still order the same dishes as when you designed the menu
🎯 **Goal**	Detect when today's market no longer looks like the one the model learned from
📓 **Notebook**	`07_monitor_drift`
🏭 **Production code**	`src/monitoring/drift.py` (last DVC stage)

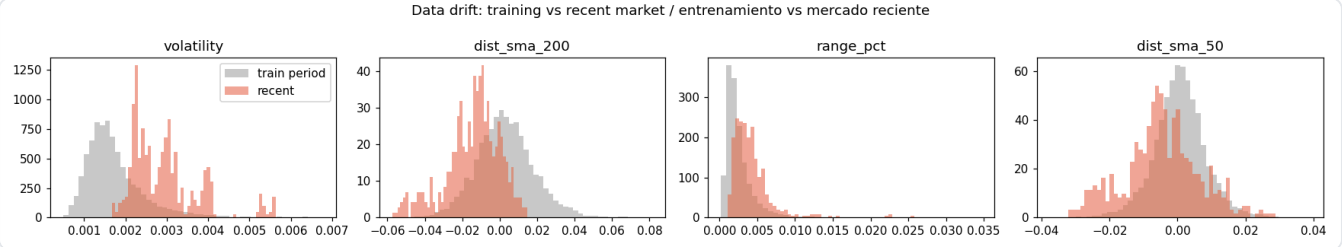
📉 Why models get worse silently



A model learns patterns from a **period**. If the market changes (new volatility regime, crisis, rally), those patterns may stop working. That's **data drift**.

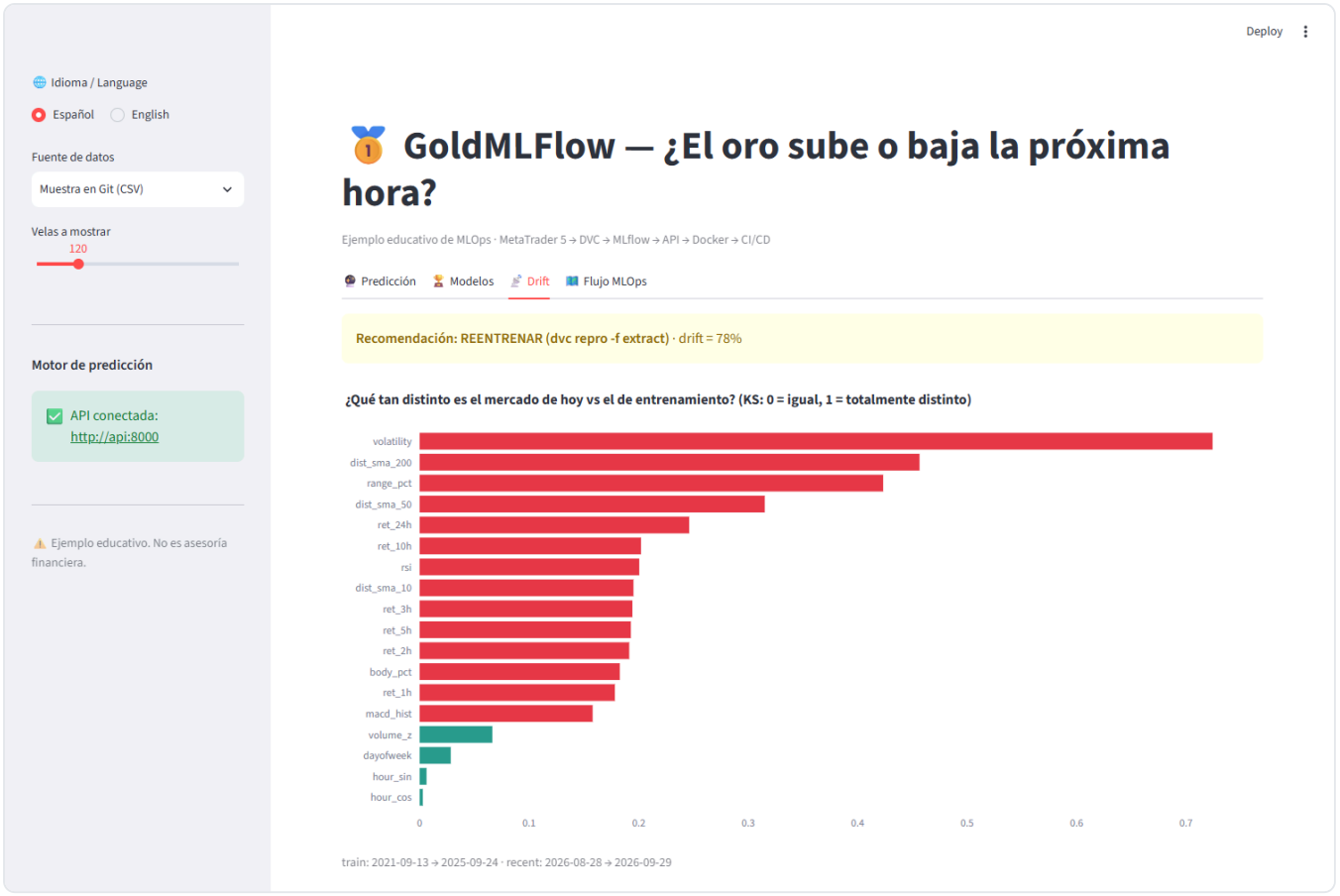
📏 How we measure it: the KS test

For each clue we compare two histograms, **training period** vs **last 500 hours**, with the **Kolmogorov–Smirnov** test: a number from **0 (identical) to 1 (totally different)**.



🔎 What we found (real data)

Clue	KS	Meaning
`volatility`	**0.72**	gold is **much** more volatile now than in most of 2021–2025
`dist_sma_200`	0.46	price is far from its long-term average (big rally/pullback)
`range_pct`	0.42	candles are bigger
`hour_sin`, `hour_cos`, `dayofweek`	~0.00–0.03	calendar clues don't drift (of course!)



🔁 What to do when it fires

```
dvc repro -f extract     # download fresh candles → the WHOLE pipeline re-runs
dvc push                 # save the new data version
git add dvc.lock models/champion reports && git commit -m "retrain: new data"
git push                 # CI checks it, CD publishes it
```

A new champion is only promoted if it **passes the quality gate**; otherwise the old one stays.

💡 Next level

- Run the drift check **on a schedule** (GitHub Actions `schedule:` or Windows Task Scheduler) and send an alert (email/Slack/Telegram).
- Also monitor **performance**: store each live prediction and compare with what really happened (the dashboard's "hit rate in window" does this visually).
- Tools: Evidently, WhyLabs, Arize, Prometheus + Grafana.

---

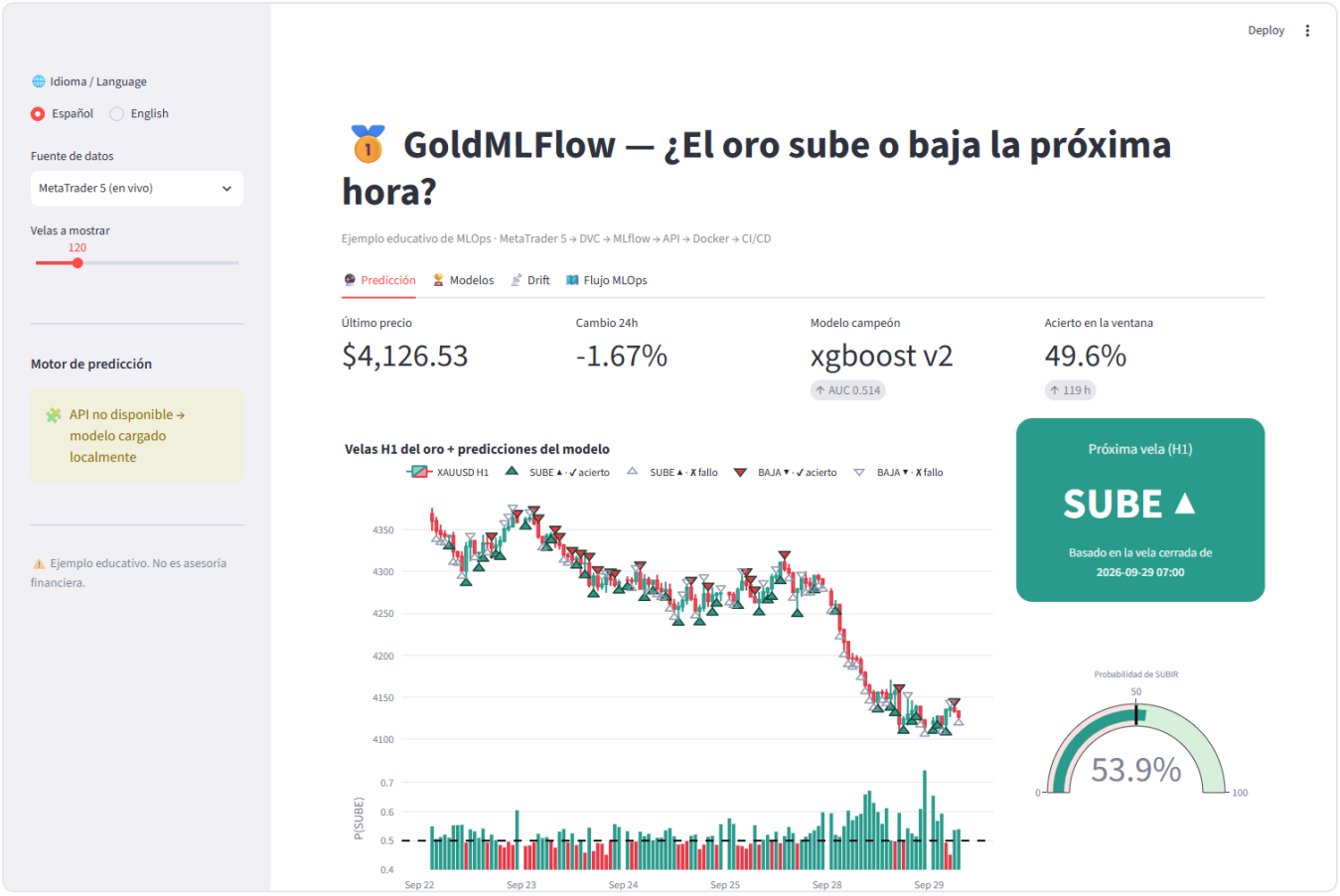
11 · Streamlit dashboard — see and use the model

🔍 **Analogy**	🪑 The dining room: where customers (non-technical people) see and enjoy the food
🎯 **Goal**	Show the whole project visually to anyone, in Spanish or English
🏭 **Code**	`app/dashboard.py`
▶️ **Run**	`streamlit run app/dashboard.py` → http://localhost:8501 · or `docker compose up`

🧭 Sidebar

Control	Options
🌐 Language	Español / English
Data source	**MetaTrader 5 (live)** (Windows only) · sample CSV in Git · upload your own CSV
Candles to show	48 – 400
Prediction engine	✅ API connected → uses the API · otherwise loads the same FastAPI app in-process

🔮 Tab 1 — Prediction



- **Cards**: last price, 24h change, champion model + AUC, **hit rate** of the model on the visible candles.
- **Big card + gauge**: the call for the **next hour** (UP ▲ / DOWN ▼) and its probability.
- **Candlestick chart**: each triangle is what the model said *before* that hour (▲ UP below the candle, ▼ DOWN above). **Coloured = right ✓, hollow grey = wrong ✗**.
- **Bottom bars**: P(UP) per hour; above the dashed 50% line → it said UP.

🏆 Tab 2 — Models

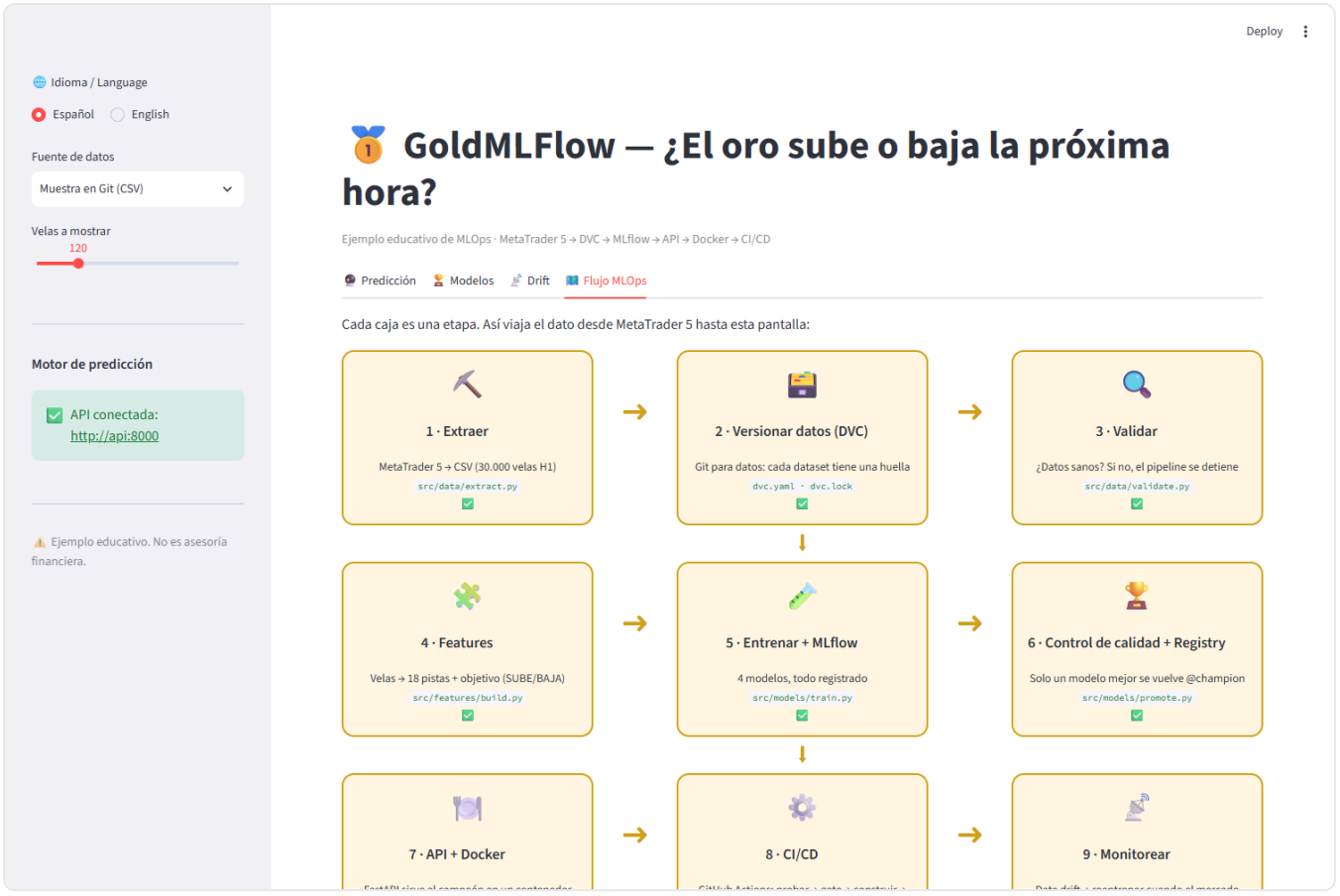


Leaderboard vs the 🪙 coin flip, the metrics table, the 🚦 quality-gate result and the toy backtest (model vs buy & hold).

📡 Tab 3 — Drift

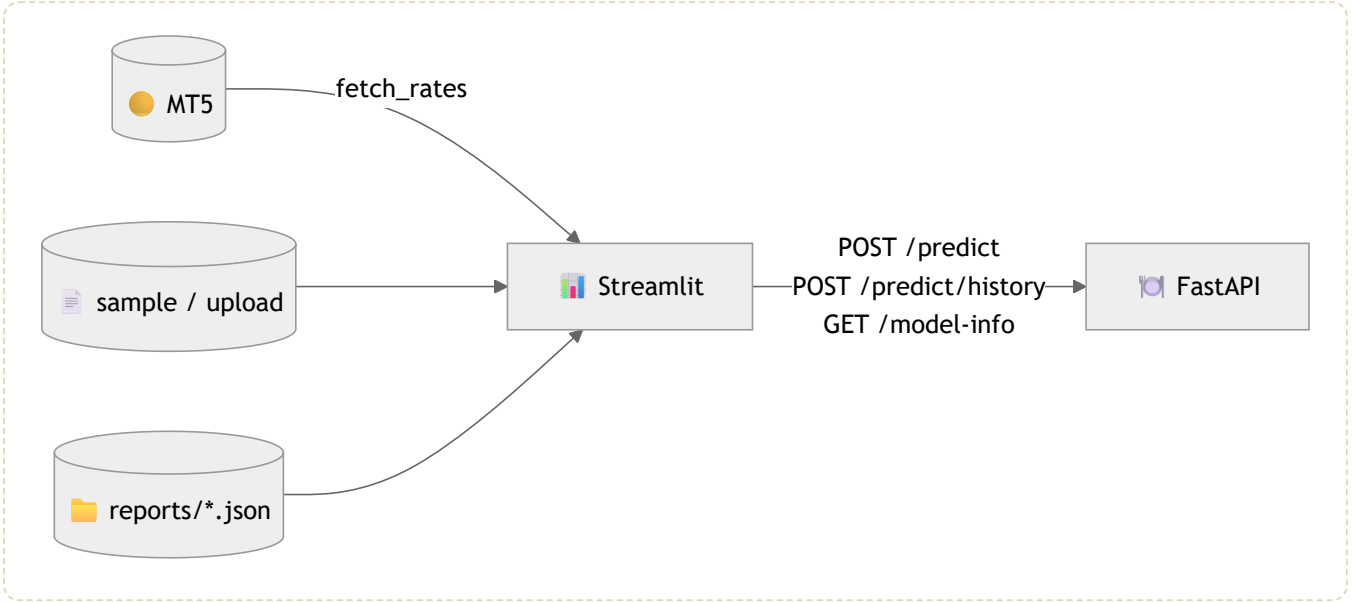
Which clues changed the most between the training period and today, plus the retrain recommendation (see chapter 10).

🗺️ Tab 4 — MLOps flow



The 9 stages as boxes: ✅ when that stage's output exists. Perfect to **explain the project to someone who knows nothing about MLOps**, box by box.

🧱 How it's built



- The dashboard is a **client** of the API (separation of concerns: UI ≠ model).

- If the API isn't running, it starts the **same** FastAPI app in-process (`TestClient`), so it never duplicates logic.
- It checks `/model-info`, not just `/health`, to make sure the thing on that port is *our* API.

---

12 · From notebook to production code

🍳 **Analogy**	🧪 A chef experiments freely in the test kitchen (notebook); the winning recipe is then written as a precise card every cook can repeat (`.py`)
🎯 **Goal**	Understand **why and how** each notebook became a production script

🔀 Two worlds, two jobs

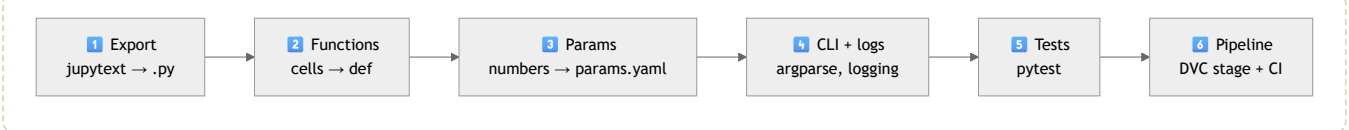
📓 Notebook	🏭 `.py` script
Discover: try, plot, make mistakes	**Repeat**: same result every time
Cells run in any order (hidden state!)	Runs top to bottom, always
Values typed by hand	Values from `params.yaml`
You look at the output	Logs, exit codes, JSON reports
Hard to test / review in Git	Unit tests, code review, CI
Run by clicking ▶	Run by DVC, CI, Docker

🗺️ The map of this project

Notebook	Became	Test
`01_extract_mt5`	`src/data/extract.py`	— (needs MT5)
`02_validate_explore`	`src/data/validate.py`	`tests/test_data.py`
`03_features`	`src/features/build.py`	`tests/test_features.py`
`04_train_mlflow`	`src/models/train.py`	CI smoke pipeline
`05_evaluate_registry`	`src/models/promote.py` + `check_gate.py`	CI quality gate
`06_serve_api`	`src/api/main.py` + `Dockerfile`	`tests/test_api.py`
`07_monitor_drift`	`src/monitoring/drift.py`	CI smoke pipeline

Every notebook ends with a "🔁 **From notebook to production**" table showing exactly what changed.

🪜 The 6 steps (do them in this order)



1️⃣ Export the notebook to a plain `.py`

```
jupytext --to py:percent notebooks/03_features.ipynb -o notebooks/exported/03_features.py
```

The results are in `notebooks/exported/`: the notebook as text, with `# %%` marking each cell. **It's a starting point, not production code.**

2️⃣ Turn cells into functions

```
# 📓 notebook cell
X["rsi"] = 100 - 100 / (1 + gain / loss)
```

```
# 🏭 src/features/build.py
def rsi(close: pd.Series, window: int) -> pd.Series:
    """Relative Strength Index (0-100)."""
    ...
```

3️⃣ Move magic numbers to `params.yaml`

```
for w in [10, 50, 200]:                # 📓 hard-coded
for w in p["sma_windows"]:             # 🏭 from params.yaml → DVC re-runs when it changes
```

4️⃣ Add a command line and logging

```
parser.add_argument("--input", default=params["features"]["processed_path"])
log.info("%d rows x %d features | saved %s", ...)
```

→ the same script works on the full data (DVC) and on the sample (CI).

5️⃣ Write tests for what you checked "by eye"

In notebook 03 we *saw* leakage in a chart → in production `test_no_leakage_features_ignore_the_future` checks it on every push.

6️⃣ Plug it into the pipeline

Add a stage to `dvc.yaml` (deps, params, outs) and, if needed, a step in `ci.yml`.

✅ Checklist: is my script production-ready?

- [ ] No hidden state: runs top to bottom in a clean terminal
- [ ] No hard-coded paths or numbers → `params.yaml`
- [ ] `if __name__ == "__main__": main()`
- [ ] Clear errors & non-zero exit code on failure
- [ ] At least one test for its critical logic
- [ ] Declared as a DVC stage with deps/outs

- [ ] Passes `ruff check`

---

13 · Glossary & FAQ

📖 Glossary (in plain words)

Term	Plain words
MLOps	Practices to take ML models from a notebook to reliable, repeatable production
Candle (OHLCV)	Summary of 1 hour of trading: Open, High, Low, Close, Volume
H1	Timeframe of 1 hour per candle
Binary classification	Predicting one of two answers: UP (1) or DOWN (0)
Feature	A numeric "clue" given to the model
Target / label	The right answer the model must learn
Baseline	The dumbest reasonable model (always UP). Anything real must beat it
Data leakage	A clue that secretly contains future information → fake great results
Overfitting	Memorizing the past (noise) instead of learning patterns
Walk-forward CV	Validation that always trains on the past and tests on what comes next
ROC-AUC	How well the model ranks UPs above DOWNs. 0.5 = coin, 1.0 = perfect
Pipeline (DVC)	Chain of stages with declared inputs/outputs; re-runs only what changed
DVC remote / storage	Where data versions live (here a local folder; in companies S3/GCS…)
Experiment tracking	Recording params, metrics and artifacts of every training run (MLflow)
Run	One training attempt recorded in MLflow
Artifact	A file saved with a run (plot, model, report)
Model signature	The model's "contract": which inputs it accepts and what it returns
Model Registry	Catalog of approved model versions
Alias (@champion)	A movable label pointing to the version in production
Quality gate	Written rules a model must pass before being promoted
API	A door through which programs talk to each other over HTTP
Endpoint	One specific door of the API (e.g. `/predict`)
Docker image / container	The packed box / the box running
Registry (ghcr.io)	Where Docker images are stored and shared
CI	Automatic checks on every change
CD	Automatic delivery of changes that passed CI
Data drift	Today's data no longer looks like the training data
KS test	Statistical test that measures how different two distributions are (0–1)

Term	Plain words
Training/serving skew	Computing features differently in training vs production (a silent bug)

❓ FAQ

Can I get rich with this model? No. It's ~51.5% right and lost to buy & hold in the test year. It's a teaching tool. Not financial advice.

`MT5 initialize() failed` Open the MetaTrader 5 terminal and log in (a demo account works). The Python package talks to the running terminal.

My broker has no `XAUUSD` Add its name to `params.yaml → data.symbol_candidates` (the extract script tells you which names it found).

I'm on Mac/Linux Everything works except extraction. Use the sample CSV (see 01 · Setup).

`dvc repro` **says "didn't change, skipping"** That's the point! Force a stage with `dvc repro -f <stage>` (e.g. `-f extract` to download fresh data).

`dvc pull` **fails on another PC** The storage is a local folder on *your* PC. For teams, use a shared/cloud remote: `dvc remote modify localstorage url s3://bucket/path`.

Port 8000 is already in use `$env:API_PORT=8010; docker compose up` (or `uvicorn src.api.main:app --port 8010`).

Why does CI train on only 4,000 candles? The GitHub robot has no MT5 and can't reach your local DVC storage. CI checks that the *code* works; the *real* model is trained on your PC and checked by the quality gate.

Where do I see the CI results? https://github.com/MirandaCR/GoldMLFlow/actions → click a run → **Summary**.

How do I roll back to an older model? In the MLflow UI (Models → gold-direction-h1) move the `@champion` alias to the older version, then re-export with `python scripts/export_champion.py`, then commit `models/champion`.
